

CareLink - LiveKit Closed-Loop Voice Escalation

Hackathon Build Specification + Coding-Agent Master Prompt

Prepared from the supplied CareLink repository and the LiveKit Telephony project screenshot. Verified against current LiveKit documentation on 29 Sep 2026.

Project audit	Result
Current CareLink stack	React/Vite + Express, Gemini/Groq, Supabase, Qdrant, Mem0-style memory, XGBoost/SHAP, agent orchestration, RAG, safety, observability
LiveKit code in supplied repo	Not present. No livekit agent package, SIP client, telephony API, or call state machine found.
Current On-Call UI	UI-only state toggle. It changes the doctor status and appointment urgency, but does not place a real call.
Screenshot telephony project	Shows a LiveKit project named "adhikaar-agent" with Calls / Dispatch rules / Phone numbers / SIP trunks. This is not proof that the CareLink repo is connected to it.

Core decision

Add the call capability, but do not add it as a generic "AI phone bot". Add it as a Closed-Loop Critical Escalation layer: detect a critical event, place an outbound call to the on-call clinician, communicate a minimal alert summary, capture acknowledgement, and escalate if acknowledgement is not received.

This adds a visible action loop to a system that already has prediction, agent reasoning, clinical evidence, safety gates, and observability. It also gives judges a concrete end-to-end moment: **critical event -> autonomous action -> human acknowledgement -> audit trail.**

1. What is already strong in your repo

The supplied zip is materially more advanced than a simple chatbot. The current codebase already contains the main agentic ingredients, so recreating them would waste hackathon time.

Area	Observed in repo	Do not rebuild
Agent orchestration	Supervisor + specialist agents + execution steps	Do not replace with another chatbot framework
Clinical RAG	Guideline corpus, retrieval endpoint, citation resolver	Do not add a second vector stack
Medication safety	Deterministic DDI checks plus LLM explanation	Keep deterministic blocker outside LLM
PHI guardrails	Anonymization / de-anonymization endpoints and agent use	Do not send raw identifiers into new external calls
Risk ML	XGBoost readmission risk + SHAP	Do not add another predictive model just for demo effect
Memory	Patient-scoped memory with fallback	Keep the same source of truth
Feedback	Approval/override + drift metric endpoints	Use call acknowledgement as another auditable event, not a second feedback system
Observability	Run traces, metrics, RAGAS evaluation	Extend the trace to telephony instead of adding a new dashboard backend

Important repo gap

There is currently no actual critical-alert event bus or escalation service. The dashboard shows telemetry and alerts, but the supplied code does not yet turn a critical event into a real outbound phone call. That gap is exactly where the LiveKit layer should attach.

Also note a baseline quality issue: running the Python test suite from the repository root in this environment stops at an import-path error in backend/api.py ("No module named model"). Before declaring the new telephony feature complete, the coding agent should first establish a clean baseline test command and then re-run the full suite after every telephony change.

2. Feature to add: Closed-Loop Critical Escalation

Do not make "call a doctor" a standalone checkbox. Make it a stateful escalation workflow.



Answer -> acknowledge / escalate -> update alert state -> audit timeline

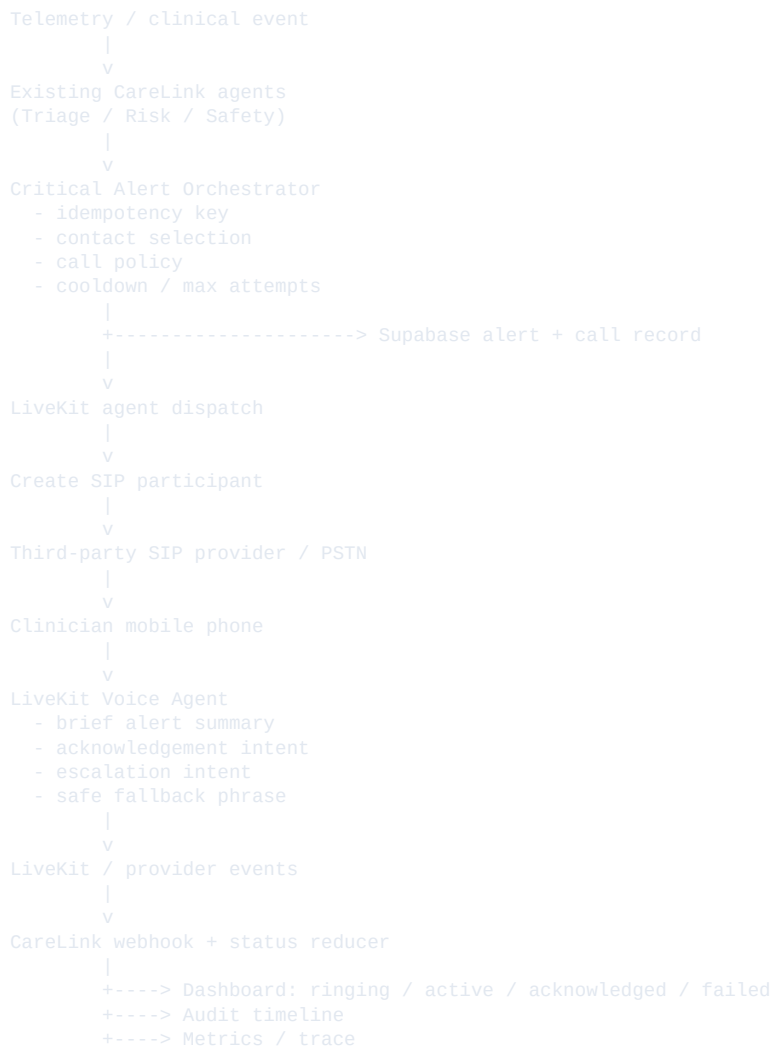
Target behavior

- **Detect:** a critical condition is emitted by the existing triage / risk layer or a future telemetry rule.
- **Decide:** the escalation orchestrator verifies this is a new, actionable critical event and selects the on-call contact.
- **Dispatch:** the backend dispatches the LiveKit voice agent and creates a SIP participant for the verified destination number.
- **Talk:** the agent gives a concise alert summary and does not invent treatment or dosages. It can read the already-computed alert facts and evidence IDs.
- **Acknowledge:** the clinician can say "acknowledge", "I am handling it", or use DTMF 1. The call ends only after a valid acknowledgement or explicit timeout/failure policy.
- **Escalate:** no answer, busy, or failed acknowledgement triggers a second contact or a visible manual escalation path.
- **Audit:** every state change is recorded with alert ID, call ID, timestamps, status, acknowledgement, and safe metadata.

What not to build

- Do not build a general-purpose voice chatbot for patients in the hackathon scope.
- Do not let the voice agent diagnose independently or recommend medication changes on the phone.
- Do not attempt arbitrary caller-ID spoofing or an "unknown number" trick. Use a verified provider number / DID and current provider rules.
- Do not make calls on every medium-risk event. Use a strict critical threshold plus idempotency and cooldown controls.

3. End-to-end technical architecture



Recommended responsibility split

Component	Responsibility	Location
Alert policy	Critical threshold, idempotency, cooldown, max attempts, escalation ladder	CareLink server
Telephony adapter	LiveKit API call placement, SIP participant creation, errors	CareLink server module
Voice agent	Realtime speech, concise alert delivery, acknowledgement handling	Separate LiveKit agent service
Call state	Queued -> ringing -> active -> acknowledged / failed / escalated / ended	Supabase
UI	Active calls, recent escalation, status, manual Call Now, test mode	React cockpit/dashboard

Why a separate voice-agent directory?

Your current repo is an Express/Vite application and does not contain LiveKit Agents code. A small **voice-agent/** package keeps the telephony runtime isolated while the main CareLink server remains the policy and data authority. LiveKit Agents supports both Python and Node.js; use Node.js here unless an existing Python LiveKit agent already works and can be reused.

4. MCP instruction for the coding agent

Use the official LiveKit Docs MCP as the coding agent's source of truth. Do not let the agent invent SDK signatures from memory. The official docs MCP endpoint is <https://docs.livekit.io/mcp>.

For Codex / compatible MCP clients

```
codex mcp add --url https://docs.livekit.io/mcp livekit-docs
```

For VS Code MCP config

```
{
  "servers": {
    "livekit-docs": {
      "type": "http",
      "url": "https://docs.livekit.io/mcp"
    }
  }
}
```

The coding agent must query the MCP for the current docs on: outbound calls, SIP trunk setup, agent dispatch, CreateSIPParticipant, call status / events, webhooks, SIP grants, and testing.

Important distinction

The **Docs MCP** is for the coding agent to inspect current LiveKit documentation. It is not the same thing as adding an MCP server to the runtime voice agent. LiveKit Agents supports runtime MCP toolsets, but the official runtime MCP documentation currently marks that feature as Python-only. Do not introduce runtime MCP into the call path unless the project intentionally uses a Python LiveKit agent.

5. Exact implementation plan for the coding agent

- 1 Baseline first**
Run the existing test suite and build. Fix only blocking baseline issues so the project has a reproducible green command.
- 2 Add dependencies**
Install the current stable LiveKit server SDK, protocol package if required by the current SDK, and the current LiveKit Agents package for Node.js. Use MCP docs to determine exact package versions and imports.
- 3 Create a telephony module**
Add server-side module(s) that create a reusable LiveKit API client and expose a single function such as `placeEscalationCall(input)`. Keep SIP credentials only on the server.
- 4 Create a call policy**
Add explicit rules: critical only, allowlisted contacts in demo mode, 1 primary attempt, optional secondary attempt, cooldown, and idempotency key per alert.
- 5 Create the voice agent**
Add a LiveKit Agent that reads only the supplied alert context, handles acknowledgement language, and refuses diagnosis or treatment changes. It should be concise enough for a real phone call.
- 6 Wire agent dispatch to SIP**
Dispatch the named agent to a deterministic room, then create the SIP participant for that room. Use a verified outbound trunk. Do not create a new trunk for every call.
- 7 Persist call state**
Create a Supabase table for escalation calls and/or alert events. Store status transitions, timestamps, contact target, LiveKit room/call ID, and an acknowledgement boolean.
- 8 Webhook / event reducer**
Add a signed LiveKit webhook endpoint and reduce participant / room events into the call state machine. Preserve raw signed payload validation before parsing.
- 9 Build the UI**
Add a compact Critical Escalation panel: contact, current status, last action, acknowledgement, retry, manual Call Now, and Test Call.
- 10 Test with a real number**
Use a dedicated test contact. Verify ringing, answer, agent audio, acknowledgement, hangup, dashboard status, and audit record.
- 11 Demo hardening**
Add mock telephony mode, deterministic fixtures, duplicate-event protection, provider failure handling, and a visible fallback manual-call button.

6. Environment variables and API contract

Keep all secrets server-side. Do not expose LiveKit API secrets, SIP auth passwords, or provider tokens to the React bundle.

Suggested environment variables

```
# LiveKit Cloud
LIVEKIT_URL=https://<your-project>.livekit.cloud
LIVEKIT_API_KEY=<server-api-key>
LIVEKIT_API_SECRET=<server-api-secret>
LIVEKIT_SIP_TRUNK_ID=<stored-outbound-trunk-id>
LIVEKIT_VOICE_AGENT_NAME=carelink-escalation-agent

# SIP / provider values, only if using inline trunk config
SIP_TRUNK_HOSTNAME=<provider-sip-host>
SIP_AUTH_USERNAME=<provider-sip-username>
SIP_AUTH_PASSWORD=<provider-sip-password>
SIP_FROM_NUMBER=<verified-did-or-exophone>

# Call policy
TELEPHONY_MODE=mock # mock | live
TELEPHONY_PRIMARY_CONTACT=+91XXXXXXXXXX
TELEPHONY_SECONDARY_CONTACT=+91XXXXXXXXXX
TELEPHONY_MAX_ATTEMPTS=2
TELEPHONY_COOLDOWN_SEC=300
TELEPHONY_RING_TIMEOUT_SEC=30
```

Suggested endpoints

Endpoint	Purpose
POST /api/telephony/escalate	Accept a critical alert event and start the policy-driven escalation flow.
POST /api/telephony/call	Manual clinician-triggered outbound call for testing or urgent action.
GET /api/telephony/calls/:id	Return current call state and non-sensitive metadata.
GET /api/telephony/calls?active=true	Dashboard list of active / recent calls.
POST /api/webhooks/livekit	Verify LiveKit signature and consume participant / room events.

Alert payload

```
{
  "alertId": "alert-2026-00031",
  "severity": "CRITICAL",
  "patientRef": "CASE-104",
  "ward": "ICU-4A",
  "bed": "12",
  "reasonCode": "SP02_CRITICAL",
  "facts": {
    "spo2": 78,
    "heartRate": 132,
    "systolic": 82,
    "trend": "rapid deterioration"
  },
  "evidenceTags": ["[NICE-CG-01]"]
}
```

7. LiveKit server-side call pattern

The current LiveKit API uses a SIP participant for outbound calls. The current docs also show the recommended ordering: dispatch the agent to the room, then create the SIP participant that joins that room. Use the exact signatures returned by the MCP at implementation time.

```
import { LiveKitAPI, SipCallError } from 'livekit-server-sdk';

const lk = new LiveKitAPI({
  host: process.env.LIVEKIT_URL!,
  apiKey: process.env.LIVEKIT_API_KEY!,
  secret: process.env.LIVEKIT_API_SECRET!,
});

export async function placeEscalationCall(input) {
  const room = `carelink-alert-${input.alertId}`;

  await lk.agentDispatch.createDispatch(
    room,
    process.env.LIVEKIT_VOICE_AGENT_NAME!,
    {
      metadata: JSON.stringify({
        alertId: input.alertId,
        severity: input.severity,
        patientRef: input.patientRef,
        ward: input.ward,
        bed: input.bed,
        reasonCode: input.reasonCode,
        facts: input.facts,
        evidenceTags: input.evidenceTags,
      }),
    },
  );

  try {
    const participant = await lk.sip.createSipParticipant(
      process.env.LIVEKIT_SIP_TRUNK_ID!,
      input.destination,
      room,
      {
        participantIdentity: `clinician-${input.alertId}`,
        participantName: 'CareLink Clinical Escalation',
        waitUntilAnswered: false,
      },
    );

    return { room, participant };
  } catch (error) {
    if (error instanceof SipCallError) {
      throw new Error(`SIP call failed: ${error.status}`);
    }
    throw error;
  }
}
```

The exact property names may change across SDK releases. That is why the coding agent must consult the LiveKit Docs MCP before finalizing the code.

Voice-agent behavior

SYSTEM ROLE

You are CareLink's clinical escalation voice agent.

RULES

1. Speak only from the alert context supplied by the server.
2. Never invent vitals, diagnoses, medications, or timestamps.
3. Do not prescribe or change treatment on the call.
4. State that this is an automated clinical alert from CareLink.
5. Ask for acknowledgement after the alert summary.
6. Accept natural-language acknowledgement and DTMF 1 if wired.
7. If the clinician asks for unsupported clinical advice, say the alert should be reviewed by the responsible clinician and do not improvise.
8. Keep the initial message under 25 seconds.

OPENING TEMPLATE

"This is CareLink automated clinical escalation. Critical alert for case CASE-104 in ICU-4A, bed 12. SpO2 is 78 percent, heart rate 132, and systolic pressure 82 with rapid deterioration. Please assess the patient immediately.

Say acknowledge when you have received this alert."

8. Safety, privacy, and demo discipline

The new phone layer creates an important contradiction with the current README claim that zero raw PHI ever leaves the local boundary. A real outbound phone call necessarily crosses a telephony boundary, and the voice agent may process information outside the original local process.

Therefore change the product claim after implementation to something precise, for example: **"CareLink minimizes external clinical data, redacts direct identifiers before model calls, and sends only the minimum alert context needed for escalation."** Do not present a stronger zero-PHI claim after introducing live telephony unless you can technically and contractually substantiate it.

Required controls

- Use minimum-necessary alert content. Prefer case reference + location + vital facts, not name, MRN, DOB, or full clinical history.
- Store SIP credentials and provider secrets only in server environment / secret manager.
- Validate every phone destination. In demo mode, restrict calls to a configured allowlist.
- Add idempotency: the same alert ID must not create duplicate simultaneous calls.
- Add cooldown and max-attempt limits to prevent call storms.
- Verify LiveKit webhook signatures before consuming events.
- Do not record calls by default. Enable recordings only when explicitly needed for the demo and policy permits it.
- Keep the voice agent informational and escalation-focused. Do not allow autonomous prescription or treatment changes.
- Use a verified caller number from the telephony provider. Do not attempt arbitrary caller-ID spoofing.

India telephony note

For an India-facing deployment, use a compliant SIP/PSTN provider workflow and provider-approved caller identity. Exotel currently documents SIP trunking for PSTN origination and requires account/KYC/credentials/phone number setup. TRAI rules distinguish transactional/service voice calls from unsolicited commercial communication, so the exact purpose and consent model still matter for production use. For a hackathon demo, use a controlled clinician test number and do not position the feature as mass outbound calling.

9. Testing plan - must be completed before the agent says "done"

Testing should prove both the happy path and the failure path. A screenshot of a call in the LiveKit dashboard is not enough.

Test	Expected result
Unit: critical gate	Only policy-approved CRITICAL alerts can trigger live calls.
Unit: idempotency	Same alert ID twice -> only one active call.
Unit: phone validation	Invalid / unapproved number rejected in demo mode.
Unit: metadata minimization	Agent metadata contains only approved alert fields.
Unit: state machine	queued -> ringing -> active -> acknowledged / failed / escalated.
Integration: LiveKit	Dispatch succeeds and SIP participant is created in expected room.
Integration: phone	Test phone rings and receives agent audio.
Integration: ack	Voice acknowledgement changes status to ACKNOWLEDGED.
Integration: no answer	Primary call times out and escalation policy runs exactly once.
Integration: provider failure	Call failure is captured with SIP/provider error and dashboard status.
Security: webhook	Invalid webhook signature is rejected.
E2E: demo flow	Simulated CRITICAL event -> real call -> ack -> audit timeline.
Regression	Existing CareLink agent, RAG, DDI, feedback, and ML tests remain green.

Commands the agent should add

```
# Baseline + regression
npm run lint
npm run build

# Telephony service tests
npm run test:telephony

# Full backend tests, from the correct project root / PYTHONPATH
python -m pytest -q backend/tests

# Manual live test only when TELEPHONY_MODE=live
npm run test:call -- --to +91XXXXXXXXXX
```

Do not hide failing baseline tests by changing test discovery. Fix the import path / package layout so the suite runs reproducibly.

10. What else should be added to the hackathon build?

Do not add ten unrelated features. Add only features that strengthen the same clinical action loop.

Feature	Purpose	Scope
Critical Escalation Timeline	Single timeline showing detected -> called -> answered -> acknowledged -> resolved.	P0
Call Acknowledgement + Escalation Ladder	Primary contact first, secondary contact only after defined failure.	P0
Manual Call Now	Let clinician trigger the same safe calling path from a patient/alert page.	P0
Mock Telephony Mode	Deterministic simulator for judging when provider/network is unavailable.	P0
Hindi + English voice mode	Switch agent language for Bharat-focused demos.	P1
SMS / email fallback	Send a short fallback notice after failed calls.	P1
FHIR export of alert	Export a structured alert/care handoff for interoperability story.	P2

The best narrative is not "we added calling". It is: CareLink detects deterioration, reasons over evidence, protects the output with a deterministic safety gate, and autonomously closes the loop by reaching the responsible clinician and capturing acknowledgement.

11. Judge-facing 90-second wow sequence

1. **1. Start with a normal patient** - Open Dashboard. Show the current risk score, telemetry, and agent cockpit.
2. **2. Trigger critical deterioration** - Use a deterministic demo fixture so the alert is reproducible. Show the critical banner and the reason code.
3. **3. Let CareLink act** - Do not click Call Now. The alert orchestrator should automatically move to CALLING and show the destination role.
4. **4. Phone rings** - Show the actual phone call. The agent identifies the alert, reads the minimum facts, and asks for acknowledgement.
5. **5. Say "acknowledge"** - The agent hears the acknowledgement and the dashboard changes to ACKNOWLEDGED.
6. **6. Show the audit trail** - Open the alert timeline. Show timestamps, call ID, call status, acknowledgement, and the agent trace.
7. **7. Show the safety angle** - Trigger a DDI case separately. The medication safety gate blocks the unsafe combination before any clinical plan is emitted.
8. **8. Close with the loop** - The point is not voice. The point is autonomous, auditable clinical escalation with a human safety checkpoint.

12. COPY-PASTE MASTER PROMPT FOR YOUR CODING AGENT

Give this entire section to the agent after it has access to the CareLink repo and LiveKit Docs MCP.

You are the implementation agent for the CareLink hackathon project.
You have the full CareLink repository. Your job is to implement a production-shaped, demo-reliable CLOSED-LOOP CRITICAL ESCALATION system using LiveKit Telephony.

NON-NEGOTIABLE PROCESS

1. Inspect the existing code before editing anything.
2. Use the official LiveKit Docs MCP at <https://docs.livekit.io/mcp> for every LiveKit SDK / telephony API detail. Do not guess method names or options.
3. Preserve the existing CareLink agents, RAG, medication safety, PHI guardrails, memory, feedback, observability, and ML behavior.
4. First make the existing project test command reproducible. Record the exact baseline command and fix blocking import/build problems before adding new code.
5. Keep all secrets server-side.
6. Work incrementally. After each logical phase, run tests and fix regressions.
7. Do not overbuild. Prefer a small complete vertical slice over many partial features.

TARGET VERTICAL SLICE

CRITICAL ALERT -> ESCALATION POLICY -> LIVEKIT AGENT DISPATCH -> SIP OUTBOUND CALL
-> VOICE ALERT SUMMARY -> HUMAN ACKNOWLEDGEMENT -> STATUS UPDATE -> AUDIT TIMELINE

REPOSITORY PLAN

- Add a server-side telephony module under the existing backend/server structure.
- Add a separate voice-agent/ directory for the LiveKit realtime agent unless an existing working LiveKit agent can be safely reused.
- Add Supabase persistence for alert/call state if the current schema supports it; otherwise create a migration/schema file with RLS-aware access.
- Extend the existing Agent Cockpit / Dashboard with a Critical Escalation panel.

LIVEKIT REQUIREMENTS

- Use a stored outbound SIP trunk when practical.
- Reuse one long-lived outbound trunk instead of creating one per call.
- Dispatch the voice agent to the deterministic room before creating the SIP participant.
- Use CreateSIPParticipant / the current JS SDK equivalent from the latest docs.
- Add a signed LiveKit webhook endpoint for participant/room lifecycle events.
- Treat SIP call failures explicitly and surface status in the UI.
- Use the provider's verified caller number. Never spoof arbitrary caller IDs.

CALL POLICY

- Only CRITICAL events may auto-call.
- Demo mode uses an allowlist of test numbers.
- Add an idempotency key based on alertId.
- Add a cooldown window and max attempt count.
- Primary contact first; secondary contact only after the configured failure conditions.
- Add a manual Call Now action for clinician testing.
- Add TELEPHONY_MODE=mock for deterministic judging and CI.

VOICE AGENT SAFETY

- The agent may read the server-supplied alert facts.
- It must never invent facts or diagnoses.
- It must not prescribe, change medication, or provide treatment instructions.
- It should say it is an automated CareLink clinical escalation.
- Keep initial alert delivery concise, ideally under 25 seconds.
- Accept natural language acknowledgement such as "acknowledge" and "I am handling it".
- Add DTMF 1 acknowledgement if the current SDK/provider path supports it.
- If asked for unsupported clinical advice, redirect to the responsible clinician.

PRIVACY

- Send minimum necessary context to external voice services.
- Do not send patient name, MRN, DOB, SSN, or full chart unless explicitly required and separately justified.
- Do not store call recordings by default.
- Do not place credentials in React/Vite environment variables prefixed for the client.
- Validate all webhook signatures before processing.
- Update README language so that privacy claims match the new telephony architecture.
- Do not claim that literally zero clinical data leaves the local boundary once live telephony is enabled unless this is actually true and documented.

UI REQUIREMENTS

- Add a compact but premium Critical Escalation component showing:
- alert severity and reason
 - destination role (not full phone number)
 - current state: queued/ringing/active/acknowledged/failed/escalated
 - elapsed time

- manual Call Now
- retry / escalate controls where policy permits
- last call result
- link/button to open the audit timeline
- a visible Test Call action when in mock mode

TESTING REQUIREMENTS

Write deterministic tests for:

- critical gate
- idempotency / duplicate alert protection
- phone normalization and allowlist
- call state transitions
- acknowledgement parsing
- no-answer escalation
- provider failure handling
- webhook signature validation
- metadata minimization
- mock telephony mode
- regression of all existing CareLink tests

E2E TEST ACCEPTANCE

The implementation is NOT complete until this works:

1. Trigger deterministic CRITICAL fixture.
2. CareLink creates exactly one escalation record.
3. Agent dispatch is created.
4. SIP participant is created.
5. Test phone rings in live mode.
6. Voice agent speaks the alert.
7. Human says acknowledgement.
8. Call becomes ACKNOWLEDGED.
9. Dashboard reflects the state.
10. Audit timeline contains timestamps and call ID.
11. Replaying the same alert does not create another active call.
12. Mock mode reproduces the same state machine without a real provider.

DOCUMENTATION REQUIREMENTS

Create/update:

- docs/TELEPHONY.md
- docs/TELEPHONY_TESTING.md
- .env.example
- README.md (only claims that are actually implemented)
- docs/TODO.md with explicit completed/pending items

DEFINITION OF DONE

- code compiles
- baseline tests pass
- telephony unit tests pass
- mock E2E passes
- real test call passes or the exact external blocker is documented
- LiveKit dashboard shows the call
- no secrets are committed
- no duplicate call can be triggered by the same alert
- final diff is focused and reversible
- provide a final implementation summary with files changed, commands run, test results, required environment variables, and any provider/KYC blockers

Do not stop at "code added". Verify the vertical slice end to end.

13. Verified references for implementation

These are the sources checked while preparing this build spec. The coding agent should still use the live documentation MCP for the final SDK signatures.

Source	URL
LiveKit outbound calls	https://docs.livekit.io/telephony/making-calls/outbound-calls/
LiveKit outbound workflow	https://docs.livekit.io/telephony/making-calls/workflow-setup/
LiveKit SIP trunk setup	https://docs.livekit.io/telephony/start/sip-trunk-setup/
LiveKit SIP API reference	https://docs.livekit.io/reference/telephony/sip-api/
LiveKit JS server SDK - SipClient	https://docs.livekit.io/reference/server-sdk-js/classes/SipClient.html
LiveKit webhooks and events	https://docs.livekit.io/intro/basics/rooms-participants-tracks/webhooks-events/
LiveKit testing telephony	https://docs.livekit.io/telephony/testing/
LiveKit Docs MCP	https://docs.livekit.io/reference/developer-tools/docs-mcp/
LiveKit Agents introduction	https://docs.livekit.io/agents/
LiveKit pricing	https://livekit.com/pricing
LiveKit quotas	https://docs.livekit.io/deploy/admin/quotas-and-limits/
Exotel SIP trunking quick start	https://docs.exotel.com/dynamic-sip-trunking/exotel-sip-trunking-quick-start
TRAI UCC overview	https://www.trai.gov.in/hi/node/10412

Final note

The most important implementation decision is scope control. Do the call layer as a closed-loop escalation workflow, not as a generic voice assistant. Then spend the remaining time on reliability, the audit trail, the UI state transitions, and a deterministic demo fixture. Those parts are visible to judges and make the feature feel like a real system rather than a one-off API integration.